The Runtime Becomes the Gatekeeper

Day 5 Final Lecture: Event Loops, Preemption, and the Capstone

Mentor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu

July 2026
Summer Course at Lanzhou University
The Hong Kong Polytechnic University (PolyU)

Table of content

The Fifth Day Closes the Loop

The view so far.
Frames with CRC (Day 1), a pub-sub bridge with QoS (Day 2), a `poll()` server handling many clients (Day 3), threads sharing a serial fd through a mutex (Day 4).

The pattern that keeps coming back.
One fd that must not be touched by two flows at once. One queue that drains at the speed of the actuator. One stop command that has to beat everything else.

Today's question.
Can we do all of this with *one thread*, no locks, and a measured worst-case latency from the operator's stop click to the wire?

1 Three I/O Styles

Blocking, the Day 3 Baseline

The shape.
Call `read()`; if there are no bytes, the thread sleeps on the socket's wait queue; when bytes arrive, the kernel wakes it.

The good.
Linear code. The next line runs only after the call returns. No callbacks, no state machines.

The bad.
One thread can sleep on exactly one fd. Day 3's slow-client lab showed where this leads, and the cure was multiplexing.

Callbacks: Split the Work in Two

The shape.
Register a function that the runtime will call when something happens. The runtime owns the thread; your code owns the response.

The mental flip.
Instead of *wait for bytes, then handle*, you write *here is what to do when bytes arrive*. Day 2's `mosquitto_message_callback_set` was this exact pattern, hidden inside a library.

The cost.
State that used to live in local variables now has to live somewhere the callback can reach: in a struct, a closure, a per-connection context object.

Event Loops: Your Own Runtime

The shape.
An infinite loop that does three things: ask the kernel which fds are ready, dispatch the registered handler for each, return to the top.

What goes inside.
A table of $fd \to handler + state$. A way to add and remove entries at runtime. Optionally, timers and signal sources alongside the sockets.

Why this wins.
One thread, predictable scheduling, no locks. The CPU spends almost all its time in the kernel waiting; when it wakes, it does exactly the work the kernel told it to.

One Thread Is Usually Enough

The mostly-blocking shape of I/O code.
Bridge between fds, route MQTT, time an e-stop: every operation either is a syscall or finishes in microseconds. The CPU is idle.

When to break the rule.
Long-running CPU work (a CRC over a megabyte, a sandbox check) belongs in a worker thread. The event loop keeps the I/O moving; the worker keeps the CPU happy.

The lesson from yesterday.
Locks exist because threads share state. One thread sharing nothing with itself needs no locks. The event-loop design is partly an exercise in not creating problems to solve.

2 epoll: What poll() Could Not Do

Where poll() Hurts at Scale

The two flaws.
Every `poll()` call passes the whole fd array *to the kernel*; the kernel scans the array *against its own data*; both walks are linear in the number of watched fds.

Why we did not feel it Day 3.
At ten clients, the walks cost microseconds. At ten thousand clients, they dominate, and a single event-loop iteration takes longer than the events between iterations.

The fix.
Give the kernel a persistent watch list, ask only "what is ready now" rather than "look at all of these". Linux's name for that is `epoll`.

epoll in Three Calls

- `epoll_create1(0)`: allocate an in-kernel watch set, get a fd back.
- `epoll_ctl(ep, ADD/MOD/DEL, fd, &event)`: register, change or remove a fd.
- `epoll_wait(ep, evs, n, timeout)`: sleep until one or more fds are ready; receive only the ready ones.

Reading the API.
The watch set survives between calls; you mutate it incrementally. The wait returns only ready events, so iteration is linear in the number that fired, not the number that exist.

The complexity result.
Each call is $O(1)$ amortised in the number of watched fds; the loop's wall-clock cost scales with *events*, not connections.

Level-Triggered vs Edge-Triggered

Level-triggered (default).
As long as a fd *still has* pending data, `epoll_wait` keeps reporting it. Same semantics as `poll()`. Forgive partial reads.

Edge-triggered (`EPOLLET`).
Report each readable event exactly once. You must `read()` in a loop until `EAGAIN`, or the rest stays parked until the next change.

When to prefer edge-triggered.
Very high event rates where the wake-up itself is expensive. For our course, stay level-triggered; partial reads are the kind of bug that wastes lab time.

An epoll Server Skeleton

```
int ep = epoll_create1(0);
struct epoll_event ev = {.events = EPOLLIN, .data.fd = listen_fd};
epoll_ctl(ep, EPOLL_CTL_ADD, listen_fd, &ev);

struct epoll_event evs[64];
for (;;) {
    int n = epoll_wait(ep, evs, 64, -1);
    for (int i = 0; i < n; i++) {
        int fd = evs[i].data.fd;
        if (fd == listen_fd) accept_new(ep);
        else                 read_one_chunk(fd);
    }
}
```

Waking the Loop: the Self-Pipe Trick

The problem.
Another thread (or a signal handler) needs to tell the event loop "drop what you are doing". The loop is sleeping inside `epoll_wait`; nothing wakes it but a watched fd.

The trick.
Create a pipe at startup; `epoll_ctl` the read end into the watch set. To wake the loop, anyone writes one byte to the write end. The loop sees `EPOLLIN`, reads the byte, and *then* checks the cancellation flag.

Modern flavours.
`eventfd` is the same idea with one fd and a counter. `signalfd` converts signals into readable bytes. All three turn external events into fds, the only thing an event loop understands.

Timers as fds, Too

`timerfd_create`.
A fd that becomes readable every N milliseconds, or once after a one-shot delay. The number of expirations is a counter you `read()`.

Why this fits the event-loop world.
Yesterday's bridge spawned a thread with `usleep` for heartbeats. With `timerfd`, the heartbeat is just another fd in the watch set, fired by the same loop that handles MQTT.

The slogan.
Everything is a fd; everything goes through `epoll_wait`. Networking, IPC, timers, signals, cancellations. One mental model.

3 Async Cancellation and Deadlines

What Cancellation Actually Means

The naive version.
"Stop". But the bridge has a queue of `MOVE` frames waiting to be written, a pending MQTT `PUBACK` in flight, a timer half-expired. Cancelling *what* from *which state*?

Cooperative cancellation.
A flag (or counter) that every long-running operation checks at well-defined points. The operation *cooperates* by stopping voluntarily; nobody yanks it from the outside.

Why pthreads' `pthread_cancel` is dangerous.
It is preemptive: it can stop a thread in the middle of holding a lock. Day 4's deadlock returns, this time from below. Stick to flags.

Signals as Soft Interrupts

What happens when SIGUSR1 arrives.
The kernel picks one of your threads, saves its registers, jumps to your handler. When the handler returns, the thread resumes the instruction it was on. Day 3's NIC interrupt story, one floor up.

The constraint on the handler.
The interrupted thread might be inside `malloc`, holding a lock you do not own. Your handler runs in *its* stack frame. So: only *async-signal-safe* calls (the list is short), and writes only to `volatile sig_atomic_t`, which the compiler treats atomically.

What `write(pipe, ...)` buys us.
Of the async-signal-safe calls, `write()` is what we use to poke the loop. Everything else, including the actual cancellation work, runs from the loop body, on its own time.

The E-Stop Pattern in C

```c
volatile sig_atomic_t estop = 0;

void wake_loop_on_estop(int sig) {
    estop = 1;
    write(wakeup_pipe[1], "x", 1);    /* unblock epoll_wait */
}

/* in the loop, after epoll_wait returns: */
if (estop) {
    drop_queued_motion_frames();      /* the cancellation */
    send_estop_frame();
    estop = 0;
}
```

Wake-Up + Flag = a Measurable Bound

Two cooperating mechanisms.
The flag tells the loop *what* to do; the pipe write tells the loop *when* to wake up to see the flag. Without the wake-up, the loop could sleep through the stop.

What you have just built.
A reliable signal path from any thread, any signal handler, any timer into the event loop, with no shared state past one atomic flag and one pipe.

The bound we can now measure.
Wake-up latency (microseconds) + queue-drain time (proportional to queue size) + transmission of one `ESTOP` frame (L/R from Day 1). Three known numbers, no hidden costs.

Deadlines, Not Timeouts

The difference.
A timeout is "wake me in 50 ms". A deadline is "be done by absolute time T". After a slow handler, a timeout drifts; a deadline does not.

Why this matters for safety.
An e-stop policy is "from operator click to motor halt within 50 ms". That is a deadline. Computing the remaining budget at each step is how you keep the guarantee under load.

Implementing it.
Record `clock_gettime(CLOCK_MONOTONIC)` at the trigger. Subtract from the deadline at each handler step. If the remainder goes negative, log a deadline miss; never silently absorb it.

In-Class Quiz: Which Wake-Up Is Safe?

Your epoll loop sleeps inside `epoll_wait`. A signal handler raises the e-stop flag. Which mechanism is the safest way to make the loop notice *within microseconds*?

1. Have the loop time out every 1 ms and poll the flag.
2. Call `pthread_cancel` on the loop's thread.
3. Write one byte to a self-pipe whose read end is in the watch set.
4. Set `O_NONBLOCK` on every watched fd.

Answer: $\boxed{3}$. The pipe turns the interrupt into fd readiness; the loop wakes immediately and performs cancellation in its own controlled context.

4 The Sub-Deadline E-Stop

Approve, Reject, Confirm

From the HCR handbook (Chapter 7).
Every motion plan passes three sandbox verdicts: *approve*, *reject*, *confirm*. Only approved frames reach the actuator; confirmed ones become an audit entry.

The physical e-stop channel.
In parallel with the bus, a hardware button drops a signal directly into the bridge. It bypasses the planner and the broker entirely.

Our reduction.
Today's lab approximates the channel with `SIGUSR1`. Same shape, no button: the handler pokes the loop, the loop drops the queue, one `ESTOP` goes out.

The Three Stages of an E-Stop

1. *Detect*: signal handler runs, sets flag, pokes the pipe. Bounded by signal-delivery latency, sub-millisecond.
2. *Drop*: loop wakes, scans the motion queue, frees pending frames. Bounded by queue length times a constant.
3. *Transmit*: loop assembles an `ESTOP` frame, hands it to the serial fd. Bounded by L/R from Day 1.

What you have to measure.
Total time from `kill(pid, SIGUSR1)` to the simulator receiving `ESTOP`. Run it under load (planner publishing at 50 Hz) and under quiet. The worst case under load is the only number that matters.

Where the Hidden Costs Hide

Suspect 1: long handlers.
If one MQTT message handler takes 8 ms, the loop cannot react for 8 ms. The worst case grows with the slowest handler.

Suspect 2: blocking work.
Any `fsync`, file write, or unbounded `write()` on a non-empty send buffer steals time from the loop. Move them to a worker thread; talk to the worker through `eventfd`.

Suspect 3: priority of the loop's thread.
Yesterday's priority inversion comes back. If the loop runs at default priority and a CPU hog runs alongside it, the loop waits its turn. Give the loop a real-time priority for the e-stop demo and document the trade-off.

Real-Time Scheduling for the Loop

Linux has more than one scheduler.
SCHED_OTHER (default) is fair-share with nice values. *SCHED_FIFO* and *SCHED_RR* are real-time classes: a SCHED_FIFO thread runs as long as it wants, preempting any normal thread on its core.

What it buys the e-stop loop.
A real-time loop wakes the moment its watched fd is ready, with no delay behind a CPU hog at default priority. Worst-case latency drops from "anywhere" to "as soon as the kernel can context-switch".

The price.
A SCHED_FIFO thread that never blocks or yields can hang its core. Use real-time priority only on threads that spend almost all their time in `epoll_wait`, and document the assumption.

5 The Gatekeeper Capstone

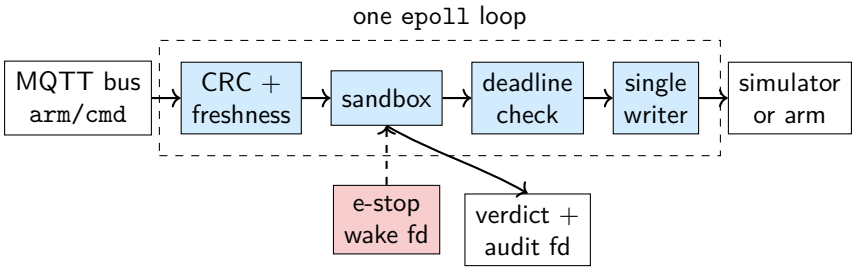
Five Gates Compose One Safe Decision

Every motion command must pass, in order.

1. *Integrity*: frame length and CRC are valid.
2. *Meaning and freshness*: opcode, units and sequence number are acceptable.
3. *Sandbox*: joint, velocity, obstacle and skin-distance constraints all pass.
4. *Single writer*: exactly one flow owns the actuator path.
5. *Deadline*: enough budget remains to put the command on the wire.

Composition rule. Any "no" becomes *reject* or *clarify*; it is audited, but it never enters the motion queue. The arm does not move on uncertainty.

The Event Loop Is Now the Gatekeeper



Preemption is part of the architecture. The MQTT socket, timer, wake pipe and writable actuator fd share one watch set. An e-stop wakes the same loop, cancels queued motion and takes the fast path to the writer.

One Command, End to End

Scenario: `MOVE(axis=z, position=30, seq=108)` arrives at QoS 1.

1. The MQTT socket becomes readable; `epoll_wait` returns it.
2. The loop reconstructs the frame; CRC, schema and sequence all pass.
3. The sandbox checks limits and clearance against the current state: *approve*.
4. The loop recomputes the absolute deadline; the wire budget still fits.
5. It records the verdict, transfers ownership to the serial writer and sends one frame.
6. The simulator returns `ACK`; the loop publishes the result upstream.

Where motion begins. Only after gates 1–5 agree. The ACK closes the trace; it does not retroactively justify the decision.

Reject Paths Are First-Class Test Fixtures

Fixture	Gate verdict	Observable evidence
Valid, clear motion	approve	one audit line + wire frame + `ACK`
One flipped payload bit	reject: integrity	CRC reason; zero serial bytes
Repeated sequence number	reject: stale	previous/current SEQ in audit
`MOVE(30)` without units	clarify: meaning	clarification publish; no queue entry
Skin-clearance breach	reject: sandbox	failed predicate + distance

The invariant across four non-happy paths. Record the decision first; never take the writer path. Separately, the e-stop fixture must preempt a 50 Hz load within the stated deadline.

Evidence Is the Handoff to Hands-On and the Book

The runtime produces three reusable artefacts.

- Wake-up and e-stop latency distribution → the chapter's timing claim.
- Fixture transcript and verdict log → the chapter's safety argument.
- Command-to-`ACK` trace → the chapter's end-to-end worked example.

Hands-on handoff. Start from the supplied gatekeeper scaffold; integrate the assigned gate, run the assigned fixture, and preserve the input, verdict, timing and output as one evidence bundle.

Writing handoff. Turn that bundle into one IGI-book subsection: method, expected invariant, measured result, failure interpretation and reproducible test command. The capstone is not a new subsystem; it is the proof that the previous four days compose.

Day 5 Summary: The Runtime Is the Safety Case

Runtime mechanics.
A persistent `epoll` watch set lets one thread dispatch sockets, timers, pipes and the actuator without rebuilding the world or sharing internal state.

Preemption.
A watched wake fd plus a cooperative flag makes the e-stop immediate; absolute deadlines expose every remaining microsecond of budget.

Composition.
Integrity, freshness, sandbox, single-writer and deadline gates form one fail-closed path. Every verdict becomes evidence.

Next: the final hands-on hour.
Integrate one assigned gate, attack it with its fixture, and carry the resulting trace into the shared chapter.

Thanks for listening

Presented by Suyu Jiang

Template: Azusa Template